

Comencé actualizando la lista de repositorios y actualizando los paquetes del sistema operativo.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo apt update -y && sudo apt upgrade -y
```

Instalé las herramientas esenciales (**curl**, **wget**, **git**) que son necesarias para descargar el instalador de Kubernetes y gestionar los archivos de la práctica.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo apt install -y curl wget git
```

Descargué y ejecuté el script de instalación de K3s (una versión ligera de Kubernetes), configurando la variable **K3S_KUBECONFIG_MODE="644"** para permitir que mi usuario pueda leer el archivo de configuración sin necesitar permisos de administrador constantemente.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ curl -sL https://get.k3s.io | K3S_KUBECONFIG_MODE="644" sh -
```

Inicié el servidor de K3s manualmente en segundo plano y añadí un comando de espera (sleep 10) para dar tiempo al clúster a inicializarse correctamente antes de empezar a trabajar.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo k3s server & sleep 10
```

Verifiqué que el clúster de K3s se había iniciado correctamente ejecutando el comando **sudo k3s kubectl get nodes**. La salida confirmó que mi nodo (a6alumno17) estaba en estado Ready, con el rol de **control-plane** y ejecutando la versión **v1.34.3**.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo k3s kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
a6alumno17    Ready    control-plane   37s   v1.34.3+k3s1
```

Procedí a descargar el binario oficial de **kubectl** utilizando **curl** para obtener automáticamente la última versión estable disponible. Esto me permitirá interactuar con el clúster utilizando la herramienta estándar de Kubernetes en lugar de depender siempre del comando prefijado de K3s.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
% Total    % Received % Xferd  Average Speed   Time    Time     Current
                                 Dload  Upload   Total   Spent    Left     Speed
100 138    100 138    0    0    765    0 --:--:-- --:--:-- --:--:--    766
100 55.8M  100 55.8M    0    0  49.7M    0  0:00:01  0:00:01 --:--:--  73.7M
```

Le di permisos de ejecución al archivo binario de kubectl que acababa de descargar y lo moví al directorio **/usr/local/bin/** para poder ejecutar el comando desde cualquier ubicación en la terminal.

Verifiqué la instalación comprobando la versión del cliente, que es la **v1.35.0**.

Configuré el entorno local para gestionar Kubernetes sin privilegios de administrador (root). Para ello, creé la carpeta **.kube**, copié el archivo de configuración generado por K3s y modifiqué sus permisos y propietario para que mi usuario actual pueda leerlo y escribir en él.

Luego ejecuté `kubectl get nodes` para confirmar que la configuración de permisos funcionó correctamente. El comando devolvió el estado del nodo a6alumno17 como Ready, lo que significa que tengo control total sobre el clúster.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo chmod +x kubectl
sudo mv kubectl /usr/local/bin/
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl version --client
Client Version: v1.35.0
Kustomize Version: v5.7.1
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ mkdir -p $HOME/.kube
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo cp /etc/rancher/k3s/k3s.yaml $HOME/.kube
/config
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo chmod 600 $HOME/.kube/config
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
a6alumno17	Ready	control-plane	2m2s	v1.34.3+k3s1

Creé el directorio principal `kubernetes-aws-practice` en mi carpeta de usuario para organizar todos los archivos de la práctica de forma ordenada y accedí a él para establecerlo como mi espacio de trabajo.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ mkdir -p ~/kubernetes-aws-practice
cd ~/kubernetes-aws-practice
```

Dentro de la carpeta del proyecto, generé un subdirectorio específico llamado app. Este directorio servirá para aislar el código fuente de la aplicación (el script de Python y el HTML) del resto de archivos de configuración de infraestructura, y entré en él para comenzar a crear los archivos.

```
alumno17@A6Alumno17:~/kubernetes-aws-practice$ mkdir -p ~/kubernetes-aws-practice/app
alumno17@A6Alumno17:~/kubernetes-aws-practice$ cd ~/kubernetes-aws-practice/app
```

Creé el archivo principal de la aplicación `app.py` utilizando un bloque de texto `(cat > ... << EOF')`. Este script en Python utiliza el framework Flask para levantar un servidor web ligero que escuchará en el puerto 5000.

En el código, definí rutas clave como `/pod-info`, la cual está programada para devolver el nombre del Pod y su IP leyendo las variables de entorno del sistema. Esto es fundamental para la práctica, ya que me permitirá visualizar más adelante qué réplica específica está respondiendo a mis peticiones y demostrar así que el balanceo de carga funciona.

Al final asigné permisos de ejecución al archivo con `sudo chmod +x app.py` para asegurar que el sistema pueda arrancar el script correctamente cuando se ejecute dentro del contenedor.

```

alumno17@A6Alumno17:~/kubernetes-aws-practice/app$ cat > app.py << 'EOF'
#!/usr/bin/env python3
from flask import Flask, jsonify, send_from_directory
import os
import socket
from datetime import datetime
import sys

app = Flask(__name__)

# Variables de entorno inyectadas por Kubernetes
POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')

@app.route('/')
def index():
    return send_from_directory('.', 'index.html')

@app.route('/pod-info')
def pod_info():
    return jsonify({
        'pod_name': POD_NAME,
        'namespace': POD_NAMESPACE,
        'hostname': socket.gethostname(),
        'timestamp': datetime.now().isoformat()
    })

@app.route('/health')
def health():
    return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200

if __name__ == '__main__':
    print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.stderr)
    app.run(host='0.0.0.0', port=5000, debug=False)
EOF

sudo chmod +x app.py

```

Este archivo contiene la estructura HTML básica y los estilos CSS necesarios para que la página se vea moderna y profesional.

En el código, definí un diseño centrado usando **flexbox** y un fondo con degradado (gradient) para mejorar la apariencia visual. Esta página será lo que veré en el navegador cuando acceda a la aplicación, y mostrará dinámicamente la información del pod que me está atendiendo.

```

alumno17@A6Alumno17:~/kubernetes-aws-practice/app$ cat index.html
<!DOCTYPE html>
<html>
<head>
  <title>Kubernetes Load Balancer</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    }
    .container {
      background: white;
      padding: 50px;
      border-radius: 10px;
      box-shadow: 0 10px 25px rgba(0,0,0,0.2);
      text-align: center;
      max-width: 500px;
    }
    h1 {
      color: #667eea;
      margin: 0 0 30px 0;
    }
    .info {
      background: #f0f0f0;
      padding: 20px;
      border-radius: 5px;
      margin: 20px 0;
    }
  }
}

```

Generé el archivo **requirements.txt** para definir las dependencias externas que necesita mi aplicación Python.

En este archivo, especifiqué explícitamente el uso de **Flask versión 3.0.0** y **Werkzeug versión 3.0.0**. Fijar estas versiones es importante para garantizar la estabilidad y evitar errores de compatibilidad inesperados cuando la aplicación se construya e instale dentro del contenedor.

```

alumno17@A6Alumno17:~/kubernetes-aws-practice/app$ cat > requirements.txt << 'EOF'
Flask==3.0.0
Werkzeug==3.0.0
EOF

```

Creé el archivo **Dockerfile**, que actúa como la receta de construcción para empaquetar mi aplicación. En este archivo definí paso a paso cómo debe crearse el entorno aislado donde se ejecutará mi código.

Especifiqué el uso de una imagen base ligera (**python:3.11-slim**) para optimizar el espacio y configuré las instrucciones para copiar mis archivos locales (**app.py**, **index.html**, **requirements.txt**) dentro del contenedor en el directorio **/app**.

Por último, añadí el comando para instalar las dependencias definidas en **requirements.txt** y establecí la instrucción de arranque (CMD) para que la aplicación se inicie automáticamente en cuanto el contenedor se ponga en marcha.

```
alumno17@A6Alumno17:~/kubernetes-aws-practice/app$ cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
# Copiar archivos
COPY requirements.txt .
COPY app.py .
COPY index.html .
# Instalar dependencias
RUN pip install --no-cache-dir -r requirements.txt
# Ejecutar app
CMD ["python", "app.py"]
EOF
```

Regresé al directorio raíz del proyecto (**~/kubernetes-aws-practice**) para comenzar con la configuración de la infraestructura de Kubernetes, separándola del código fuente de la aplicación que estaba en la carpeta **app**.

Creé el archivo **namespace.yaml** para definir un espacio de trabajo aislado llamado **load-balancer-demo**.

Apliqué la configuración del clúster ejecutando **kubectl apply -f namespace.yaml**. El sistema confirmó la acción con el mensaje **namespace/load-balancer-demo configured**, lo que indica que el espacio de trabajo ya está listo para recibir los despliegues.

```
alumno17@A6Alumno17:~/kubernetes-aws-practice$ cd ~/kubernetes-aws-practice
alumno17@A6Alumno17:~/kubernetes-aws-practice$ cat > namespace.yaml << 'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: load-balancer-demo
  labels:
    name: load-balancer-demo
EOF

# Aplicar el cambio
kubectl apply -f namespace.yaml
namespace/load-balancer-demo configured
```

Generé el archivo **configmap.yaml** para crear un objeto de tipo ConfigMap llamado **app-files**. Este recurso es fundamental en Kubernetes para desacoplar los datos de configuración de la imagen del contenedor.

Dentro de la sección **data**, incrusté directamente el código fuente de **app.py** y el archivo **requirements.txt**. Al hacer esto, estoy preparando el terreno para "inyectar" estos archivos dentro de los pods como si fueran volúmenes, lo que me permite gestionar el código de la aplicación directamente desde la configuración del clúster sin necesidad de reconstruir la imagen Docker para cada cambio pequeño.

```

alumno17@A6Alumno17:~/kubernetes-aws-practice$ cat > configmap.yaml << 'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-files
  namespace: load-balancer-demo
data:
  requirements.txt: |
    Flask==3.0.0
    Werkzeug==3.0.0
  app.py: |
    #!/usr/bin/env python3
    from flask import Flask, jsonify, send_from_directory
    import os
    import socket
    from datetime import datetime
    import sys
    app = Flask(__name__)
    POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
    POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')
    @app.route('/')
    def index():
        return send_from_directory('.', 'index.html')
    @app.route('/pod-info')
    def pod_info():
        return jsonify({
            'pod_name': POD_NAME,
            'namespace': POD_NAMESPACE,
            'hostname': socket.gethostname(),
            'timestamp': datetime.now().isoformat()
        })
    @app.route('/health')

```

Ejecuté el comando `kubectl apply -f configmap.yaml` para cargar efectivamente la configuración en el clúster. Este paso es el que guarda mis archivos de código dentro de la base de datos de Kubernetes para que luego puedan ser inyectados en los contenedores.

Realicé una comprobación con `kubectl get configmap` dentro de mi **namespace** para verificar que el recurso se había creado correctamente. La salida confirma la existencia de **app-files** y la columna **DATA** indica que contiene los archivos que definí, listos para ser utilizados.

```

alumno17@A6Alumno17:~/kubernetes-aws-practice$ # Aplicar el ConfigMap
kubectl apply -f configmap.yaml

# Verificar
kubectl get configmap -n load-balancer-demo
configmap/app-files unchanged
NAME      DATA      AGE
app-files  3          71s

```

Redacté el archivo **deployment.yaml**, que es la pieza central de la arquitectura en Kubernetes. En este archivo definí cómo debe ejecutarse y mantenerse mi aplicación dentro del clúster.

Establecí el número de réplicas iniciales en 3 (replicas: 3). Esto asegura que desde el primer momento habrá tres copias idénticas de mi servidor web funcionando simultáneamente, lo cual es esencial para poder demostrar luego el balanceo de carga y garantizar que la aplicación siga disponible si uno de los pods falla.

En la especificación del contenedor, definí que debe usar la imagen `python:3.11-slim` y configuré el comando de arranque para que, al iniciarse, instale las dependencias (`pip install`) y ejecute el servidor (`python app.py`). Esto automatiza completamente el ciclo de vida de la aplicación.

```
alumno17@A6Alumno17:~/kubernetes-aws-practice$ cat > deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: load-balancer-demo
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: web-app
        image: python:3.11-slim
        command: ["sh", "-c"]
        args:
        - |
          cd /app
          pip install --no-cache-dir -r requirements.txt > /dev/null 2>&1
          python app.py
```

Verifiqué el estado inicial de los pods justo después de crear el despliegue. El comando `kubectl get pods` me mostró que las 3 réplicas se habían iniciado correctamente y ya estaban en estado Running, lo que confirma que la definición del Deployment era correcta.

```
alumno17@A6Alumno17:~/kubernetes-aws-practice$ kubectl get pods -n load-balancer-demo
NAME                                READY   STATUS    RESTARTS   AGE
web-app-674f4b5b5b-cdm7m           1/1     Running   0           78s
web-app-674f4b5b5b-lrhfn           1/1     Running   0           78s
web-app-674f4b5b5b-scdkx           1/1     Running   0           78s
```

Para garantizar que el sistema fuera robusto y no fallara en los siguientes pasos, ejecuté el comando `kubectl wait`. Esto pausó la terminal hasta que Kubernetes confirmó explícitamente (condition met) que los 3 pods estaban totalmente listos para recibir tráfico, evitando así errores por intentar acceder a la aplicación antes de tiempo.

```
alumno17@A6Alumno17:~/kubernetes-aws-practice$ echo "Esperando a que los pods estén listos..."
kubectl wait --for=condition=ready pod -l app=web-app -n load-balancer-demo --timeout=120s
Esperando a que los pods estén listos...
pod/web-app-674f4b5b5b-cdm7m condition met
pod/web-app-674f4b5b5b-lrhfn condition met
pod/web-app-674f4b5b5b-scdkx condition met
```

Realicé una última comprobación de estado pasado un tiempo. Esto me sirvió para asegurar que los pods se mantenían estables y no se reiniciaban solos.

```
alumno17@A6Alumno17:~/kubernetes-aws-practice$ kubectl get pods -n load-balancer-demo
```

NAME	READY	STATUS	RESTARTS	AGE
web-app-674f4b5b5b-cdm7m	1/1	Running	0	41m
web-app-674f4b5b5b-lrhfn	1/1	Running	0	41m
web-app-674f4b5b5b-scdkx	1/1	Running	0	41m

Generé el archivo **service.yaml** para definir cómo se expondrá mi aplicación al exterior. Este es el componente que actúa como puerta de entrada para el tráfico de los usuarios.

Especifiqué que el tipo de servicio sea **LoadBalancer**. Esta configuración es clave porque indica a Kubernetes que debe provisionar una IP única que repartirá automáticamente el tráfico entrante entre los distintos pods disponibles, sin que yo tenga que configurar manualmente a qué réplica enviar cada petición.

Configuré el mapeo de puertos para que el servicio escuche en el puerto estándar web (**80**) y redirija el tráfico internamente al puerto **5000**, que es donde mi aplicación **Flask** está escuchando dentro de los contenedores.

Añadí explícitamente la opción **sessionAffinity: None**. Esto es vital para la práctica, ya que fuerza al balanceador a no casarse con ningún pod, garantizando que cada nueva petición pueda caer en un servidor diferente y así demostrar visualmente el balanceo de carga.

```
alumno17@A6Alumno17:~/kubernetes-aws-practice$ cat > service.yaml << 'EOF'
```

```
apiVersion: v1
kind: Service
metadata:
  name: web-app-service
  namespace: load-balancer-demo
  labels:
    app: web-app
spec:
  type: LoadBalancer
  selector:
    app: web-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
      name: http
      sessionAffinity: None
EOF
```

Apliqué la configuración del servicio en el clúster ejecutando **kubectl apply -f service.yaml**. El sistema respondió confirmando que el recurso web-app-service había sido creado correctamente.

Inmediatamente después, verifiqué el estado del servicio con el comando **kubectl get svc**. En la salida, observé que el servicio aparecía con el tipo **LoadBalancer** y una IP interna asignada, aunque el campo **EXTERNAL-IP** se mostraba como pending. Esto es normal en este punto, ya que el balanceador de carga está en proceso de aprovisionamiento.


```

alumno17@A6Alumno17:~/kubernetes-aws-practice$ kubectl apply -f service.yaml
service/web-app-service created
alumno17@A6Alumno17:~/kubernetes-aws-practice$ kubectl get svc -n load-balancer-demo
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
web-app-service     LoadBalancer  10.43.53.124   <pending>      80:31907/TCP     4s

```

Ejecuté el comando `kubectl port-forward` para crear un túnel directo desde mi máquina local hacia el servicio dentro del clúster. Esto es necesario para poder probar la aplicación desde mi navegador, ya que en este entorno de desarrollo el LoadBalancer no asigna una IP pública accesible de inmediato.

Configuré la redirección del puerto **8080** de mi ordenador al puerto **80** del servicio web-app-service. La terminal mostró el mensaje `Forwarding from...`, confirmando que el túnel está activo y redirigiendo el tráfico correctamente hacia el puerto **5000** de los contenedores.

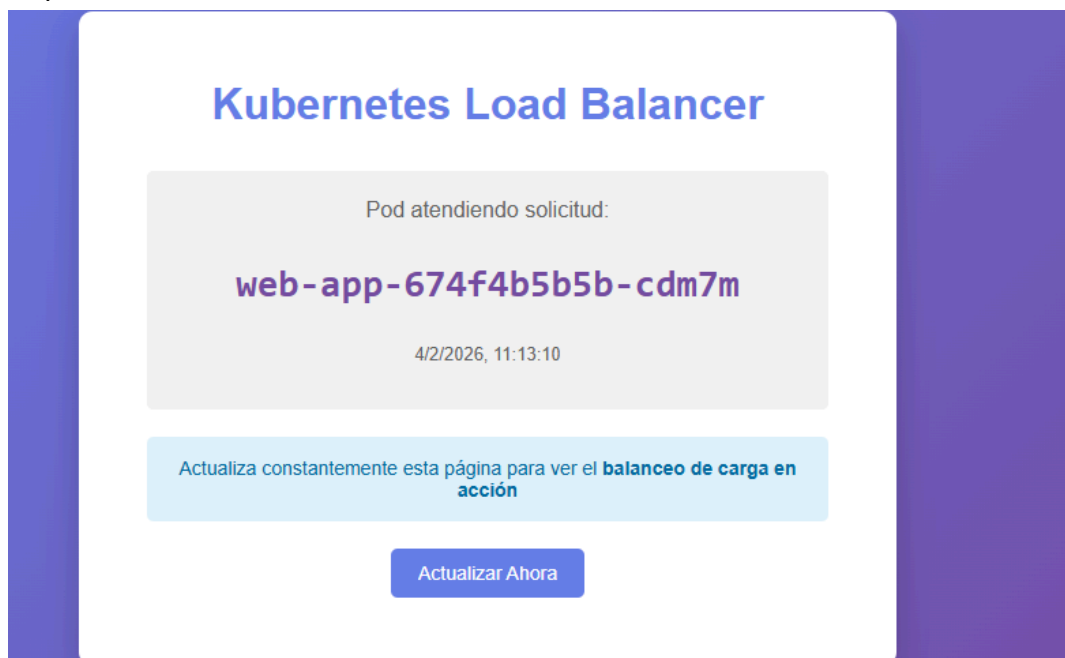
```

alumno17@A6Alumno17:~/kubernetes-aws-practice$ kubectl port-forward -n load-balancer-demo svc/web-app-service 8080:80
Forwarding from 127.0.0.1:8080 -> 5000
Forwarding from [::1]:8080 -> 5000
Handling connection for 8080

```

Abrí mi navegador web y accedí a la dirección **http://localhost:8080** para verificar el funcionamiento de la aplicación.

La interfaz cargó correctamente mostrando el diseño visual que definí en el HTML. Lo más importante es que la página me informa dinámicamente de qué contenedor está procesando mi solicitud; en este caso, veo que fue el pod **web-app-674f4b5b5b-cdm7m** el que respondió a las 11:13:10.



Hice clic en el botón Actualizar Ahora unos segundos después, a las 11:13:38, para intentar forzar una nueva petición al servidor.

Kubernetes Load Balancer

Pod atendiendo solicitud:

web-app-674f4b5b5b-cdm7m

4/2/2026, 11:13:38

Actualiza constantemente esta página para ver el **balanceo de carga en acción**

Actualizar Ahora

Estos registros aparecieron coincidiendo con mis interacciones en el navegador. Cada vez que actualizaba la página web para probar el balanceo, el túnel capturaba esa petición y la redirigía al clúster, dejando constancia aquí. Esto me confirma visualmente que el túnel está activo y funcionando, procesando exitosamente todas las solicitudes que hago desde mi entorno local hacia la aplicación en Kubernetes.

[illegible]

```
alumno17@A6Alumno17:~/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ for i in {1..10}; do
    echo "Petición $i:"
    curl -s http://localhost:8080/pod-info | python3 -m json.tool | grep pod_name
    sleep 1
done
Petición 1:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
Petición 2:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
Petición 3:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
Petición 4:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
Petición 5:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
Petición 6:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
Petición 7:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
Petición 8:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
Petición 9:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
Petición 10:
    "pod_name": "web-app-674f4b5b5b-cdm7m",
```

Detalles

Nombre del grupo de seguridad
 kubernetes-aws-sg

ID del grupo de seguridad
 sg-0abf05bd1c8a93870

Descripción
 Security group para Kubernetes con AWS

ID de la VPC
[vpc-07241f2707e798b92](#)

Propietario
 168760605961

Número de reglas de entrada
 3 Entradas de permisos

Número de reglas de salida
 1 Entrada de permiso

Reglas de entrada

Reglas de salida

Compartiendo

Asociaciones

Reglas de entrada (3)

Administrar etiquetas

Editar reglas de entrada

☐	Name	ID de la regla del gr...	Versión de IP	Tipo
☐	-	sgr-0f8222dd87b7d4312	IPv4	HTT
☐	-	sgr-04a79ea9bf6a66268	IPv4	SSH
☐	-	sgr-09b2a6fc6a97db363	IPv4	HTT

Preparé mi entorno local para la conexión segura creando el directorio `~/.ssh`, que es el estándar donde se almacenan las claves de acceso en Linux.

Copié el archivo de la clave privada (`kubernetes.pem`) desde mi carpeta de Descargas de Windows hacia este nuevo directorio en WSL para tenerlo accesible desde la terminal.

Modifiqué los permisos del archivo de la clave ejecutando `chmod 400`

```
alumno17@A6Alumno17:~/kubernetes-aws-practice$ mkdir -p ~/.ssh
alumno17@A6Alumno17:~/kubernetes-aws-practice$ cp /mnt/c/Users/*/Downloads/kubernetes.pem ~/.ssh/
alumno17@A6Alumno17:~/kubernetes-aws-practice$ chmod 400 ~/.ssh/kubernetes.pem
```

Ya conectado dentro de la máquina virtual de AWS, `ejecuté sudo apt update`. Esto lo hice para actualizar la lista de repositorios del servidor remot.

```
ubuntu@ip-172-31-76-56:~$ sudo apt update
```

A continuación, instalé las herramientas esenciales (`curl`, `wget`, `python3`, `git`) en el servidor de la nube.

```
ubuntu@ip-172-31-76-56:~$ sudo apt install -y curl wget python3 python3-pip git
```

Creé un directorio de trabajo llamado `~/kubernetes-test` dentro de la instancia de AWS utilizando el comando `mkdir -p`. Este paso es importante para mantener ordenado el entorno remoto, ya que será la ubicación donde guardaré los scripts de prueba para validar el balanceo de carga.

```
ubuntu@ip-172-31-76-56:~$ mkdir -p ~/kubernetes-test
```

Desde mi terminal local, establecí un túnel SSH inverso ejecutando el comando con la `opción -R 8888:localhost:8080`. Esta acción conecta el puerto `8888` del servidor remoto de AWS directamente con el puerto `8080` de mi ordenador, permitiendo que la máquina en la nube acceda a mi aplicación local como si estuviera en su propia red.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ ssh -i ~/.ssh/kubernetes.pem -N -R 8888:localhost:8080 ubuntu@3.238.229.87
```

Una vez dentro de la terminal de AWS, verifiqué la conexión ejecutando `curl` contra el puerto del túnel (`localhost:8888`). La respuesta JSON que recibí, que incluye el nombre del pod y el timestamp, confirma que el tráfico está viajando desde la nube hasta mi clúster local a través del túnel seguro.

```
ubuntu@ip-172-31-76-56:~$ curl -s http://localhost:8888/pod-info
{"hostname": "web-app-674f4b5b5b-cdm7m", "namespace": "load-balancer-demo", "pod_name": "web-app-674f4b5b5b-cdm7m", "timestamp": "2026-02-04T11:39:04.929876"}
```

Creé el script `test-kubernetes-lb.sh` directamente en la instancia de AWS para automatizar las pruebas de carga y no tener que ejecutar los comandos manualmente uno por uno.

Programé el script para que realice un bucle de 15 peticiones consecutivas a `localhost:8888`, que es el puerto conectado a mi clúster local vía el túnel SSH. Para cada petición, utilicé un comando de Python integrado que analiza la respuesta JSON y extrae el nombre exacto del pod (`pod_name`) que está atendiendo la solicitud.

Incluí una lógica de conteo al final del script para generar un resumen estadístico automático. Esto es fundamental para ver claramente si el tráfico se está distribuyendo entre los distintos pods disponibles o si todas las peticiones están siendo atendidas por el mismo, permitiéndome validar el funcionamiento del balanceador desde la nube.

```
ubuntu@ip-172-31-76-56:~$ cat > ~/test-kubernetes-lb.sh << 'EOF'
#!/bin/bash

echo "=== Prueba Kubernetes Load Balancer desde AWS ==="

echo ""

declare -A pods_count

for i in {1..15}; do

    response=$(curl -s http://localhost:8888/pod-info)

    pod_name=$(echo $response | python3 -c "import sys, json; print(json.load(sys.stdin)['pod_name'])" 2
    >/dev/null)

    if [ -z "$pod_name" ]; then
        pod_name="ERROR"
    fi

    echo "Petición $i -> Pod: $pod_name"

    pods_count[$pod_name]=$(( ${pods_count[$pod_name]:-0} + 1 ))

    sleep 1
done

echo ""
echo "=== Resumen Balanceo ==="

for pod in "${!pods_count[@]}"; do
    echo "Pod $pod: ${pods_count[$pod]} peticiones"
done
EOF
```

Otorgué permisos de ejecución al archivo con el comando `chmod +x` y lancé el script `./test-kubernetes-lb.sh` para iniciar la prueba automatizada de conectividad desde la instancia de AWS.

Observé en el resumen final que las 15 peticiones fueron procesadas exitosamente, lo que valida que el túnel SSH funciona correctamente.

```
ubuntu@ip-172-31-76-56:~$ chmod +x ~/test-kubernetes-lb.sh
./test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===

Petición 1 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 2 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 3 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 4 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 5 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 6 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 7 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 8 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 9 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 10 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 11 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 12 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 13 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 14 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 15 -> Pod: web-app-674f4b5b5b-cdm7m

=== Resumen Balanceo ===
Pod web-app-674f4b5b5b-cdm7m: 15 peticiones
```

Escalé la capacidad de mi aplicación ejecutando el comando `kubectl scale` para aumentar el número de réplicas de 3 a 5. Esto simula una respuesta ante un aumento de tráfico, provisionando inmediatamente más instancias para repartir la carga de trabajo.

Verifiqué el estado de los nuevos recursos con `kubectl get pods`.

Para asegurar la estabilidad antes de realizar más pruebas, volví a utilizar `kubectl wait`. El sistema esperó hasta confirmar que las 5 réplicas cumplieran la condición ready, garantizando que toda la flota estaba lista para recibir peticiones.

```
alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl scale deployment web-app -n load-balancer-demo --replicas=5
deployment.apps/web-app scaled
alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get pods -n load-balancer-demo
NAME                                READY   STATUS    RESTARTS   AGE
web-app-674f4b5b5b-668xt           0/1     Running   0           5s
web-app-674f4b5b5b-cdm7m           1/1     Running   0           84m
web-app-674f4b5b5b-lrhfn           1/1     Running   0           84m
web-app-674f4b5b5b-scdkx           1/1     Running   0           84m
web-app-674f4b5b5b-xwc2x           0/1     Running   0           5s
alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl wait --for=condition=ready pod -l app=web-app -n load-balancer-demo --timeout=120s
pod/web-app-674f4b5b5b-668xt condition met
pod/web-app-674f4b5b5b-cdm7m condition met
pod/web-app-674f4b5b5b-lrhfn condition met
pod/web-app-674f4b5b5b-scdkx condition met
pod/web-app-674f4b5b5b-xwc2x condition met
```

Volví a ejecutar el script de pruebas `./test-kubernetes-lb.sh` inmediatamente después de escalar el despliegue a 5 réplicas, con la intención de ver cómo se repartía el tráfico entre los nuevos pods.

```
ubuntu@ip-172-31-76-56: ~$ ./test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===

Petición 1 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 2 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 3 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 4 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 5 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 6 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 7 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 8 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 9 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 10 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 11 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 12 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 13 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 14 -> Pod: web-app-674f4b5b5b-cdm7m
Petición 15 -> Pod: web-app-674f4b5b5b-cdm7m

=== Resumen Balanceo ===
Pod web-app-674f4b5b5b-cdm7m: 15 peticiones
```


Antes de ver los logs, confirmé con `kubectl get pods` que las 5 réplicas seguían en estado Running, notando claramente la diferencia de edad entre los pods originales (87 minutos) y los nuevos (3 minutos).

Al revisar la salida del comando `kubectl logs`, confirmé que el servidor **Flask** se había iniciado correctamente en el nuevo contenedor, mostrando los mensajes de arranque estándar y la advertencia de servidor de desarrollo.

Observé una secuencia continua de peticiones **GET /health** con código 200 provenientes de la IP 10.42.0.1.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get pods -n load-balancer-demo
NAME                                READY   STATUS    RESTARTS   AGE
web-app-674f4b5b5b-668xt           1/1     Running   0           3m11s
web-app-674f4b5b5b-cdm7m           1/1     Running   0           87m
web-app-674f4b5b5b-lrhfn           1/1     Running   0           87m
web-app-674f4b5b5b-scdkx           1/1     Running   0           87m
web-app-674f4b5b5b-xwc2x           1/1     Running   0           3m11s
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl logs -n load-balancer-demo web-app-674f4b5b5b-668xt
[web-app-674f4b5b5b-668xt] Iniciando servidor Flask...
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.42.0.15:5000
Press CTRL+C to quit
10.42.0.1 - - [04/Feb/2026 11:50:25] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:30] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:35] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:37] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:39] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:44] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:47] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:49] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:54] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:57] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:50:59] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:51:04] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:51:07] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:51:07] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:51:12] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:51:15] "GET /health HTTP/1.1" 200 -
```

Ejecuté el comando `kubectl logs` utilizando el selector de etiquetas `-l app=web-app` junto con la opción `-f`. Esta técnica es muy potente porque me permite agregar y visualizar en tiempo real los registros de todas las réplicas simultáneamente en una sola pantalla, en lugar de tener que inspeccionar cada pod por separado.

```
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl logs -n load-balancer-demo -l app=web-app -f
10.42.0.1 - - [04/Feb/2026 11:54:36] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:39] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:42] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:44] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:47] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:52] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:54] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:57] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:55:02] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:55:04] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:34] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:38] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:39] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:42] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 11:54:47] "GET /health HTTP/1.1" 200 -
```

Ejecuté los comandos `kubectl top pods` y `kubectl top nodes` para realizar un análisis de consumo de recursos en el clúster. Comprobé que mi aplicación es muy ligera, ya que cada uno de los 5 pods consume apenas 1 o 2 milicores de CPU y 24 MiB de memoria, y el nodo principal tiene capacidad de sobra (solo 13% de RAM en uso).

```
^Calumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl top pods -n load-balancer-demo
NAME                                CPU(cores)   MEMORY(bytes)
web-app-674f4b5b5b-668xt           1m           24Mi
web-app-674f4b5b5b-cdm7m           1m           24Mi
web-app-674f4b5b5b-lrhfn           1m           24Mi
web-app-674f4b5b5b-scdkx           2m           24Mi
web-app-674f4b5b5b-xwc2x           1m           24Mi
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl top nodes
NAME                                CPU(cores)   CPU(%)       MEMORY(bytes)  MEMORY(%)
a6alumno17                          126m         0%           1086Mi         13%
alumno17@A6Alumno17:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl get events -n load-balancer-demo
LAST SEEN   TYPE      REASON          OBJECT                                          MESSAGE
7m13s       Normal    Scheduled        pod/web-app-674f4b5b5b-668xt                 Successfully assigned load
-balancer-demo/web-app-674f4b5b5b-668xt to a6alumno17
7m12s       Normal    Pulled           pod/web-app-674f4b5b5b-668xt                 Container image "python:3.
11-slim" already present on machine
7m12s       Normal    Created          pod/web-app-674f4b5b5b-668xt                 Created container: web-app
7m12s       Normal    Started          pod/web-app-674f4b5b5b-668xt                 Started container web-app
7m13s       Normal    Scheduled        pod/web-app-674f4b5b5b-xwc2x                 Successfully assigned load
-balancer-demo/web-app-674f4b5b5b-xwc2x to a6alumno17
7m12s       Normal    Pulled           pod/web-app-674f4b5b5b-xwc2x                 Container image "python:3.
11-slim" already present on machine
7m12s       Normal    Created          pod/web-app-674f4b5b5b-xwc2x                 Created container: web-app
7m12s       Normal    Started          pod/web-app-674f4b5b5b-xwc2x                 Started container web-app
7m13s       Normal    SuccessfulCreate  replicaset/web-app-674f4b5b5b                 Created pod: web-app-674f4
b5b5b-668xt
7m13s       Normal    SuccessfulCreate  replicaset/web-app-674f4b5b5b                 Created pod: web-app-674f4
b5b5b-xwc2x
45m         Normal    EnsuringLoadBalancer  service/web-app-service                     Ensuring load balancer
45m         Normal    AppliedDaemonSet     service/web-app-service                     Applied LoadBalancer Daemo
nSet kube-system/svc-lb-web-app-service-9d142bd6
7m13s       Normal    ScalingReplicaSet   deployment/web-app                           Scaled up replica set web-
app-674f4b5b5b from 3 to 5
```

Ejecuté el comando `kubectl describe deployment` para realizar una auditoría profunda de la configuración actual de mi aplicación. Esta herramienta me proporcionó una radiografía completa, mostrándome desde la imagen de Docker utilizada hasta los volúmenes montados y las sondas de salud configuradas.

Confirmé en la sección de estado (Replicas) que el clúster ha alcanzado la convergencia deseada: hay 5 available de 5 desired. Esto certifica que el proceso de escalado se completó sin errores y que las 5 instancias están totalmente operativas.

Revisé la sección de Events al final de la salida, donde el sistema registró cronológicamente la acción de escalado. Finalmente, consulté el historial de versiones con `kubectl rollout history`, observando que sigo en la **REVISION 1**, lo que indica que es la primera versión estable desplegada.


```

alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl describe deployment web-app -n load-b
alancer-demo
Name: web-app
Namespace: load-balancer-demo
CreationTimestamp: Wed, 04 Feb 2026 11:19:02 +0100
Labels: app=web-app
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=web-app
Replicas: 5 desired | 5 updated | 5 total | 5 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=web-app
  Annotations: kubectl.kubernetes.io/restartedAt: 2026-02-04T11:24:27+01:00
  Containers:
    web-app:
      Image: python:3.11-slim
      Port: 5000/TCP
      Host Port: 0/TCP
      Command:
        sh
        -c
      Args:
        cd /app
        pip install --no-cache-dir -r requirements.txt > /dev/null 2>&1
        python app.py
  Liveness: http-get http://:5000/health delay=15s timeout=1s period=10s #success=1 #failure=3
  Readiness: http-get http://:5000/health delay=5s timeout=1s period=5s #success=1 #failure=3
  Environment:
    POD_NAME: (v1:metadata.name)
    POD_NAMESPACE: (v1:metadata.namespace)
  Mounts:
    /app from app-volume (rw)
  Volumes:
    app-volume:
      Type: ConfigMap (a volume populated by a ConfigMap)
      Name: app-files
      Optional: false
      Node-Selectors: <none>
      Tolerations: <none>
  Conditions:
    Type Status Reason
    ----
    Progressing True NewReplicaSetAvailable
    Available True MinimumReplicasAvailable
  OldReplicaSets: <none>
  NewReplicaSet: web-app-674f4b5b5b (5/5 replicas created)
  Events:
    Type Reason Age From Message
    ----
    Normal ScalingReplicaSet 8m25s deployment-controller Scaled up replica set web-app-674f4b5b5b fr
om 3 to 5
alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl rollout history deployment web-app -n
load-balancer-demo
deployment.apps/web-app
REVISION CHANGE-CAUSE
1 <none>

```

Para finalizar la práctica y dejar el entorno limpio, ejecuté el comando **kubectl delete namespace load-balancer-demo**. Esta es la forma más eficiente de eliminar todo lo que he hecho, ya que al borrar el espacio de nombres, Kubernetes elimina automáticamente en cascada todos los recursos que contenía (el Deployment, los 5 Pods, el Servicio y el ConfigMap), sin necesidad de borrarlos uno por uno.

Realicé una verificación final con **kubectl get namespaces** para asegurarme de que la eliminación fue exitosa. Como se observa en la lista, el namespace load-balancer-demo ya

no aparece, quedando únicamente los espacios de nombres por defecto del sistema (default, kube-system, etc.), lo que confirma que el clúster ha vuelto a su estado original.

```
alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl delete namespace load-balancer-demo
namespace "load-balancer-demo" deleted

alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl delete namespace load-balancer-demo
namespace "load-balancer-demo" deleted

alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get namespaces
NAME                STATUS    AGE
default              Active   139m
kube-node-lease      Active   139m
kube-public          Active   139m
kube-system          Active   139m
```

Accedí a la consola de gestión de AWS para detener la instancia **kubernetes-test-server** que había creado. Este paso es fundamental para controlar los costes en la nube, asegurándome de que dejo de pagar por el tiempo de cómputo inmediatamente después de haber finalizado mis pruebas de conectividad remota.

Detener instancia

Detener la instancia le permite reducir los costos, modificar la configuración y solucionar problemas.

ID de la instancia 	Detener la protección	Resultado 
 i-0ebb516900260bf45 (kubernetes-test-server)	desactivado	 Se puede detener

► Recursos asociados

Se le cobrarán cargos por estos recursos mientras la instancia esté detenida



Se facturarán los recursos asociados

Después de detener la instancia, ya no se le cobrarán tarifas de uso o transferencia de datos. Sin embargo, se le seguirán facturando las direcciones IP elásticas y los volúmenes de EBS asociados.

Omitir el apagado del sistema operativo

Esta opción omite el proceso de apagado controlado del sistema operativo. Utilícela solo cuando deba detener su instancia inmediatamente, como durante una emergencia o una conmutación por error.

☐ Omitir el apagado del sistema operativo

Cancelar

Detener

Finalmente, procedí a limpiar mi entorno de desarrollo local para liberar recursos. Primero detuve el servicio del clúster con `sudo systemctl stop k3s` y, a continuación, ejecuté el script `k3s-uninstall.sh` para eliminar completamente Kubernetes de mi máquina, dejándola limpia y lista para futuros proyectos.

```
alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo systemctl stop k3s
[sudo] password for alumno17:
alumno17@A6Alumno17: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo /usr/local/bin/k3s-uninstall.sh
```